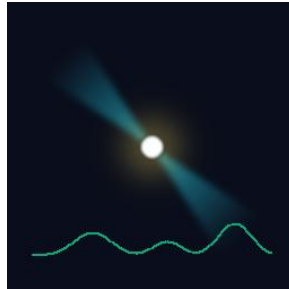


B210 Spectrum Analyzer

Release Workflow



Version 1.0.0

Richard M Hambly (KOGD)

DSES

May 2026

Contents

Contents.....	2
B210 Spectrum Analyzer — Release Workflow	4
1. Development environment	4
2. Source layout.....	4
3. Distribution host.....	5
4. Routine release workflow	5
4.1 Bump the version.....	6
4.2 Commit the source changes	6
4.3 Rebuild the user-facing install guide.....	6
4.4 Rebuild this document (optional)	6
4.5 Build the release zip	6
4.6 Upload to the server	6
4.7 Update the manifest	7
4.8 Done	7
5. The update manifest.....	7
5.1 Manifest format.....	7
5.2 Server side: writing the manifest.....	8
5.3 Behavior notes.....	8
5.4 Disabling and rolling back.....	8
6. Testing a release	9
6.1 Verify the manifest fetch and version compare	9
6.2 Verify the notification dialog renders	9
6.3 Verify the download URL.....	9
7. Reference	9
7.1 Files the release script bundles.....	9
7.2 Where the SigMF sample comes from	10
7.3 Documentation toolchain.....	10
7.4 Multi-radio architecture.....	11
7.5 Memory / notes worth knowing.....	11

B210 Spectrum Analyzer — Release Workflow

Version 1.0.0 Author: Richard M Hambly (K0GD) — rick@cncssys.com Audience: developer only — not shipped to recipients.

This document is for the developer (you) cutting and publishing new releases. It complements `DSES_RFI_Spectrum_Analyzer_Installation.pdf`, which is the user-facing install guide. Topics covered here:

- How the project is laid out and what the build tools need.
- The end-to-end routine for shipping a new release.
- How the in-app auto-update notification works and what to put on the server.
- How to test a release before announcing it.
- Reference: distribution host, file naming conventions, source-code constants.

1. Development environment

All development happens in the project-local conda environment at `./.conda`. `CLAUDE.md` captures the project rules:

- This env is created from `environment.yml` with `conda env create --prefix ./.conda -f environment.yml`.
- Dependency changes go in `environment.yml`. No global installs, no shell rc edits.
- The Qt binding is **PySide6** (commercial Qt license + LGPL friendlier for distribution). Never import from `PyQt5` / `PyQt6`.

The runtime is **Radioconda** (installed by each end user). The `./.conda` env mirrors a superset of what's needed for development plus the doc-build chain (`python-docx`, `pywin32`, `docx2pdf`).

To rebuild `./.conda` from scratch:

```
conda env create --prefix ./.conda -f environment.yml
```

or to update an existing one:

```
conda env update --prefix ./.conda -f environment.yml --prune
```

2. Source layout

Path	Purpose
<code>b210_spectrum_analyzer.py</code>	The application. Single hand-polished Python file. <code>APP_VERSION</code> near the top is the source of truth for the release version.
<code>launcher.bat</code> / <code>.ps1</code>	Windows entry point + launcher logic (finds Radioconda).
<code>launcher.sh</code>	Linux / macOS launcher.

<code>install-shortcut.ps1</code>	Windows desktop / Start-menu shortcut installer.
<code>b210-spectrum-analyzer.desktop</code>	Linux desktop entry template.
<code>icons/b210.{ico,png}</code>	Windows / Linux icons.
<code>icons/generate-icon.py</code>	Source for the icon. Re-run only if you change the design.
<code>sample.sigmf-data + sample.sigmf-meta</code>	Bundled playback sample. Loaded by the app when no B210 is attached.
<code>LICENSE</code>	GPL-3.0 full text + project copyright.
<code>environment.yml</code>	Conda env spec (dev).
<code>Installing.md</code>	Source for the user-facing install guide.
<code>Release_Workflow.md</code>	This document.
<code>build_install_docx.py</code>	Builds both .docx and .pdf from a Markdown source. CLI-parameterized (<code>--title</code> , <code>--subtitle</code> , <code>--docx</code> , <code>--pdf</code>).
<code>make-release.ps1 / .sh</code>	Stages the runtime files into <code>dist/b210-spectrum-analyzer-<version>/</code> and zips them.
<code>CLAUDE.md</code>	Project ground rules for Claude Code sessions. Not shipped.

Not part of the release bundle: `.conda/`, `.git/`, `.vscode/`, `dist/`, `CLAUDE.md`, `Installing.md`, `Release_Workflow.md`, `build_install_docx.py`, `make-release.{ps1,sh}`, `icons/generate-icon.py`.

3. Distribution host

All distribution goes through:

https://gpstime.com/sw_distribution/b210_sa/

That single directory holds:

- `manifest.json` — the version manifest the running app polls.
- `b210-spectrum-analyzer-<version>.zip` — one zip per published release. Keep at least the current + previous version online.
- `b210-spectrum-analyzer-<version>.sha256` — optional SHA-256 next to each zip so recipients can verify their download.
- (Optional) `DSES_RFI_Spectrum_Analyzer_Installation.pdf` — latest install guide, separate from the zips, for users who want to read before downloading the bundle.

There is no per-OS variant — one zip works on Windows, Linux (any current distro), macOS Intel, and macOS Apple Silicon. The OS-specific launchers ride along inside the bundle.

4. Routine release workflow

When you're ready to ship a new version:

4.1 Bump the version

Edit APP_VERSION near the top of b210_spectrum_analyzer.py. Use semantic versioning: MAJOR.MINOR.PATCH. Bug fixes only → bump PATCH; new features → bump MINOR; breaking changes → bump MAJOR.

4.2 Commit the source changes

Note the version in the commit message.

4.3 Rebuild the user-facing install guide

```
.conda\python.exe build_install_docx.py
```

That reads Installing.md and produces:

- DSES_RFI_Spectrum_Analyzer_Installation.docx (intermediate; editable in Word for spot-checks)
- DSES_RFI_Spectrum_Analyzer_Installation.pdf (the deliverable)

Skip this step if you didn't change install-relevant behavior, but err on the side of rebuilding so the version stamps inside the PDF stay current.

4.4 Rebuild this document (optional)

```
.conda\python.exe build_install_docx.py Release_Workflow.md ^  
  
--docx DSES_RFI_Spectrum_Analyzer_Release_Workflow.docx ^  
  
--pdf DSES_RFI_Spectrum_Analyzer_Release_Workflow.pdf ^  
  
--subtitle "Release Workflow"
```

Only if you've edited this file. It doesn't ship.

4.5 Build the release zip

```
Windows: .\make-release.ps1
```

```
Unix:     ./make-release.sh
```

The script reads APP_VERSION, creates dist\b210-spectrum-analyzer-<version>\ with the runtime files, and zips it to dist\b210-spectrum-analyzer-<version>.zip. The staging directory is kept so you can inspect the contents before publishing.

4.6 Upload to the server

From a shell with scp:

```
scp dist/b210-spectrum-analyzer-<version>.zip you@gpstime.com:/path/to/sw_distribution/b210_sa/
```

Then SSH in and:

```
cd /path/to/sw_distribution/b210_sa/
```

```
sha256sum b210-spectrum-analyzer-<version>.zip > b210-spectrum-analyzer-<version>.sha256
chmod 644 b210-spectrum-analyzer-<version>.{zip,sha256}
```

Verify the URL is reachable from outside:

```
curl -sI https://gpstime.com/sw_distribution/b210_sa/b210-spectrum-analyzer-<version>.zip |
head -1

# expect "HTTP/2 200" or "HTTP/1.1 200 OK"
```

4.7 Update the manifest

Edit `manifest.json` on the server with the new `latest_version`, `download_url`, and `release_notes`. Format and behavior in §5 below.

4.8 Done

Existing users running the previous release (with auto-update enabled and a working manifest URL configured) see the notification on next launch — within 24 hours of relaunch, since the program throttles checks to one per 24 hours.

5. The update manifest

The program polls a single JSON file (the “manifest”) to learn about new releases. The URL is baked into the default settings as:

```
https://gpstime.com/sw_distribution/b210_sa/manifest.json
```

End users can override it in their own `settings.ini` (`manifest_url` = under `[updates]`), e.g. to point at a staging copy for testing before publishing to all users.

5.1 Manifest format

JSON object with these fields:

```
{
  "latest_version": "1.0.1",
  "download_url": "https://gpstime.com/sw_distribution/b210_sa/b210-spectrum-analyzer-1.0.1.zip",
  "release_notes": "Added auto-update check.\nFixed recording freeze at 25 MHz.\n"
}
```

- ``latest_version`` — required. Dotted version string. The program compares this to its own `APP_VERSION` using tuple-of-ints semantics.
- ``download_url`` — required (unless you want the Open button greyed out). The dialog opens this URL in the user’s browser.

- ``release_notes`` — optional. Plain text rendered in a scrollable panel. Use `\n` for line breaks.

5.2 Server side: writing the manifest

From your SSH session, after uploading the new zip:

```
cat > /path/to/sw_distribution/b210_sa/manifest.json << 'EOF'

{
    "latest_version": "1.0.1",

    "download_url": "https://gpstime.com/sw_distribution/b210_sa/b210-spectrum-analyzer-1.0.1.zip",

    "release_notes": "Added X.\nFixed Y.\n"
}

EOF

chmod 644 /path/to/sw_distribution/b210_sa/manifest.json
```

The single-quoted `<< 'EOF'` is important so the shell doesn't expand `$/backslashes` inside the JSON.

5.3 Behavior notes

- The auto-check runs at app launch on a background thread; failures are silent. Manual Help → Check for Updates... shows an error dialog on failure.
- Throttled to one fetch per 24 hours per user. `last_check_iso` in their `settings.ini` records the last successful check.
- If the user clicks **Skip this version**, the version is saved to `dismissed_version` and they aren't shown the dialog again until you publish a strictly newer one. Bug-fix releases for the same `latest_version` don't re-prompt — bump the version if you want to re-notify.
- TLS cert: Python's `urllib.request` verifies by default. `gpstime.com` needs a normal (Let's Encrypt / commercial) cert. Self-signed will fail silently.
- Server doesn't need to serve `application/json` MIME type, but it's polite.

5.4 Disabling and rolling back

- To **revoke** a release (security issue, etc.): lower `latest_version` in the manifest back to the previous good version. Running clients on the bad version won't be told to update, but you'd still have to ask them out-of-band to roll back.
- To **disable** auto-update entirely for everyone: serve an empty manifest URL (or a 404). Users will silently fail the check at launch.
- To **disable** for a single user: have them set `auto_check = false` under `[updates]` in their `settings.ini`, or set `manifest_url = (empty)`.

6. Testing a release

6.1 Verify the manifest fetch and version compare

Before announcing a release, sanity-check the manifest with the same-version test — running app at v1.0.0, manifest also says v1.0.0:

```
curl -sS https://gpstime.com/sw_distribution/b210_sa/manifest.json  
  
# expected: the JSON you uploaded
```

In the running program, **Help → Check for Updates...** should show “You’re running the latest version (1.0.0). Manifest reports latest = 1.0.0.” That confirms TLS handshake + HTTP fetch + JSON parse + version compare all work.

6.2 Verify the notification dialog renders

Temporarily edit `manifest.json` on the server to advertise a higher version than what’s running. Use **Help → Check for Updates...** (auto-check is debounced — wipe `last_check_iso` in `settings.ini` to retrigger that path). The non-modal update dialog should appear with the release notes you wrote.

Don’t forget to flip the manifest back to the real `latest_version` before walking away, or v1.0.0 users will see a fake notification.

6.3 Verify the download URL

```
curl -sI https://gpstime.com/sw_distribution/b210_sa/b210-spectrum-analyzer-<version>.zip |  
head -1
```

Should be `HTTP/... 200`. If not, the **Open Download Page** button in the dialog will 404 in users’ browsers.

7. Reference

7.1 Files the release script bundles

`make-release.ps1` (Windows) and `make-release.sh` (Unix) produce identical bundles. Contents:

```
b210-spectrum-analyzer-<version>/  
├─ b210_spectrum_analyzer.py  
├─ LICENSE  
├─ launcher.bat  
├─ launcher.ps1  
├─ launcher.sh  
├─ install-shortcut.ps1  
└─ b210-spectrum-analyzer.desktop
```

```

├─ icons/b210.ico
├─ icons/b210.png
├─ environment.yml
├─ DSES_RFI_Spectrum_Analyzer_Installation.pdf
├─ sample.sigmf-data
└─ sample.sigmf-meta

```

Release_Workflow.md, the matching PDF, CLAUDE.md, Installing.md, build_install_docx.py, the make-release.* scripts themselves, the conda env, .git, and IDE configs are all excluded by name.

7.2 Where the SigMF sample comes from

The bundled sample.sigmf-{data,meta} is a real B210 recording at 10 MHz centered on 408 MHz (the pulsar preset). It loops in playback mode and lets users without a B210 see the GUI work.

To replace it with a different recording:

1. In live mode, hit **Record** and let it run for a few seconds.
2. Stop recording. The new file lands in
~/Documents/B210_Recordings/DSES_Spectrum_Analyzer_<timestamp>.sigmf-{data,meta}.
3. Rename to sample.sigmf-data and sample.sigmf-meta and drop next to
b210_spectrum_analyzer.py in your project root, replacing the existing pair.
4. The next make-release picks up the new sample.

Be aware the file size is dominated by the sample (several hundred MB at 10 MHz). For email-class distribution, ship a 1–2 second sample (~10–20 MB) instead, or omit it from the bundle and host it separately.

7.3 Documentation toolchain

Tool	Used for	Installed via
python-docx	Build the styled DOCX from Markdown.	environment.yml (conda-forge)
pywin32	Drive Word via COM to update fields and SaveAs PDF.	environment.yml (conda-forge, Windows-only)
docx2pdf	Listed in environment.yml as a fallback; not used by the current build script.	environment.yml (pip section)

All three are dev-only — they don't ship and the runtime (Radioconda) doesn't need them.

The build script (build_install_docx.py) accepts CLI options so the same code produces both this document and the install guide:

```
.conda\python.exe build_install_docx.py [source.md] \
    [--docx OUT.docx] [--pdf OUT.pdf] \
    [--title "Cover Title"] [--subtitle "Cover Subtitle"]
```

Defaults match the install-guide build, so a bare `build_install_docx.py` invocation builds `Installing.md` → `DSES_RFI_Spectrum_Analyzer_Installation.{docx,pdf}` with `Installation Guide` as subtitle.

7.4 Multi-radio architecture

The application drives any of:

- **UHD B200-family:** B200, B210. Via `uhd.usrp_source`, wrapped in `UhdB200Source`.
- **SoapySDR-supported devices:** SDRPlay (RSP1A / RSP1B / RSPduo / RSPdx), RTL-SDR, HackRF, Airspy, Airspy HF+, BladeRF, LimeSDR, PlutoSDR. Via `gnuradio.soapy.source`, wrapped in `SoapyGenericSource`.

The split-out classes live next to each other in `b210_spectrum_analyzer.py`:

```
RadioSource          - abstract: gr block + set_samp_rate / set_center_freq / set_gain
    UhdB200Source      - UHD path
    SoapyGenericSource - gr-soapy path; per-driver capabilities in SOAPY_DEFAULTS
```

`find_all_radios() = find_b200_uhd() + find_soapy_devices()`. The picker shows them in a single list. Settings hold `(device_driver, device_serial)`.

Per-driver capabilities (sample-rate options, gain range) come from the `SOAPY_DEFAULTS` table at module top. Add a new Soapy backend by listing its driver string and adding an entry there.

SDRPlay-specific: SDRPlay devices need both the SDRplay API installer (from sdrplay.com) and the `soapy_sdr-module-sdrplay` conda package. Neither ships in Radioconda by default. The install guide's §1A documents the steps for users.

7.5 Memory / notes worth knowing

A few project facts that don't fit elsewhere:

- The B210 USRP serial used during development is **3273A91**. New users with their own B210s get auto-detected; no source change needed (see §8 of the install guide).
- macOS users on Apple Silicon must install the **arm64** Radioconda build; mixing an `x86_64` Radioconda with `arm64` Qt produces confusing errors at startup.
- On Linux, Radioconda's udev rules must be activated for the B210 (see install guide §2.2). SDRPlay devices on Linux similarly need the udev rules from the SDRplay API installer.

- The auto-update check is harmless if the manifest URL is unreachable — failures are silent in the background path. Don't panic if you accidentally delete the manifest; users just don't see new-version notifications until you restore it.